System Hardware

SOEN228

Lab Report #3

Student Name	Student ID	Lab section	Lecture section
Michaël Gugliandolo	40213419	C-Z (lab D)	U
Jean-Nicolas Sabatini-Ouellet	40207926	B	U

Department of Computer Science & Software Engineering

Concordia University

Winter 2022

Objectives **2**

Introduction **2**

Experimental Results and Questions **3**

Discussion **7**

Conclusions **8**

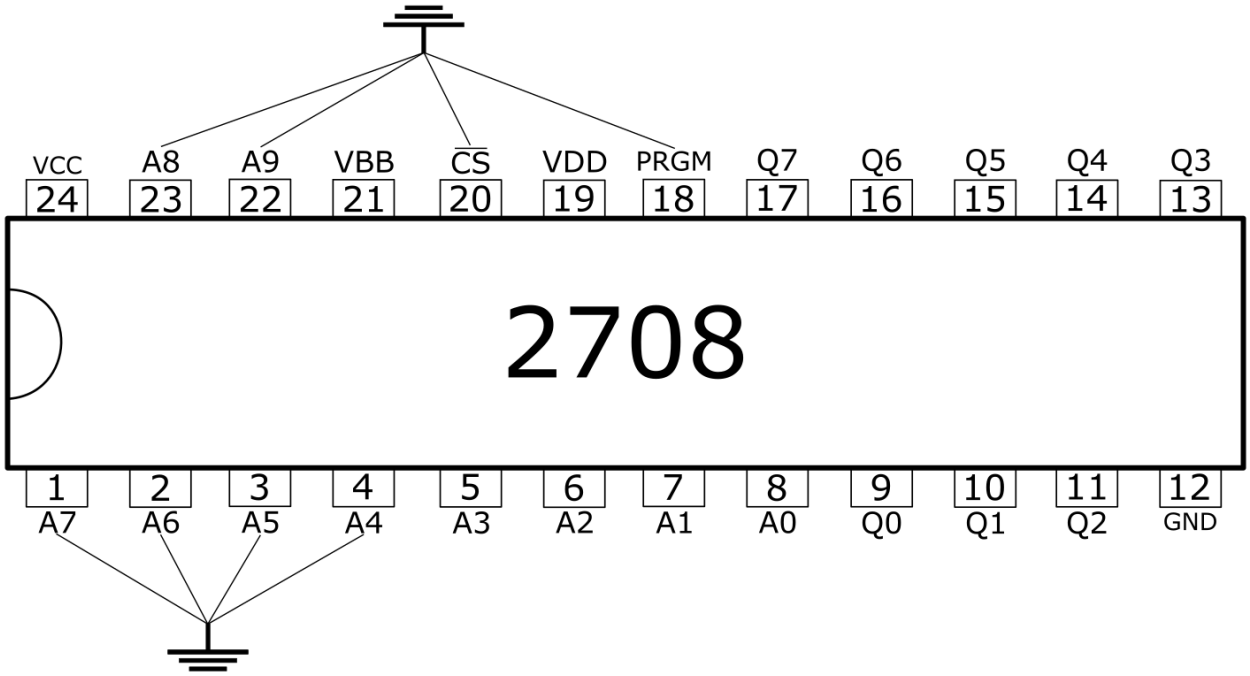
1. Objectives

This lab consists of adding data registers and a memory address register that will be used to access the content of an EPROM chip. The values outputted by the ROM are instructions in binary. We will feed these numbers into a decoder to separate each instruction into its own line.

2. Introduction

In this lab, we will use three 74HC574 8-bit registers for the data registers A and B, and the memory address register (MAR). We will convert them to 4-bit registers by grounding the last 4 input bits. In addition, they have an output enable (OE) pin to control when the register can output its value; it can only output when its OE has a value of 0. They also have a positive-edge clock pin, so when we change its value from logic 0 to logic 1, the register will read the value from the bus and update its internal flip-flops.

The 2708 EPROM chip is a read-only memory (ROM) chip with 10 input pins for the address and 8 output pins for the stored values. Since our computer is only 4-bit, we have to ground the 6 highest inputs pins.



The ROM chip outputs numbers/instructions in binary. To utilize the instructions, we have to use the 74154 decoder so that each instruction will activate a single line. This way, we can do something different for each control line; this means that a decoder takes m inputs and has 2^m outputs. Decoders are either active low or active high. When they're active low, the chosen line

(based on input) will be logic 0 while the other lines will be logic 1, and vice versa for an active high decoder. For example, this is how a 2 input bit decoder works:

Input	Active high				Active low			
00	**1**	0	0	0	**0**	1	1	1
01	0	**1**	0	0	1	**0**	1	1
10	0	0	**1**	0	1	1	**0**	1
11	0	0	0	**1**	1	1	1	**0**

In the 74154 decoder, pins E0 and E1 have to be grounded.

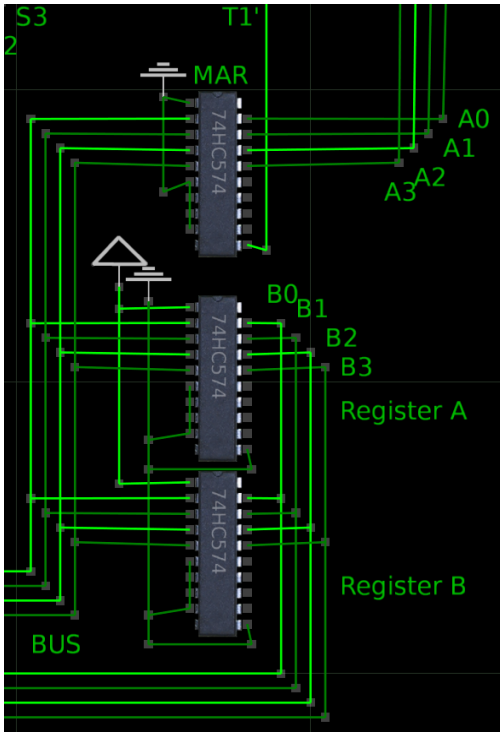
Furthermore, the numbers representing instructions are called op-codes. For example, if the decoder outputs 11011 (so the third instruction), it represents the op-code 0x2 (in hex), which means MovAB (ie. move value from register A to B). Our computer will have 5 possible instructions:

Op-codes (in hex)	Instruction
0x0	IncA (increment register A)
0x1	IncB (increment register B)
0x2	MovAB (move/copy value from A to B)
0x3	MovBA (move/copy value from B to A)
0x4	NOP (no operation)

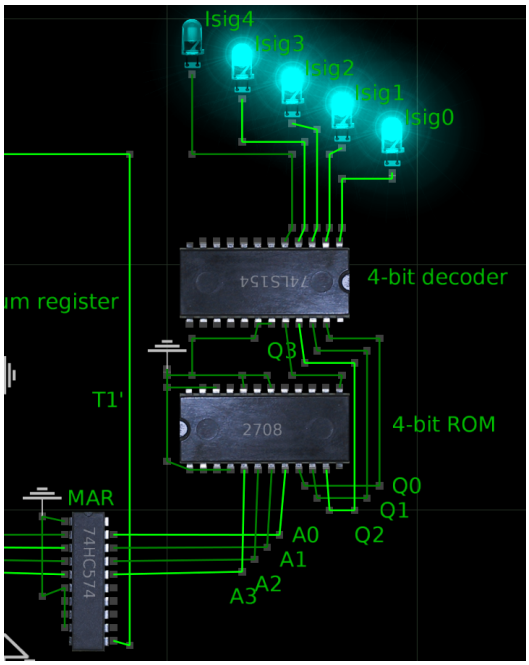
3. Experimental Results and Questions

In this lab, the OE of registers A and B are connected to logic 1 so they never output and their clocks are grounded so they never read from the bus. The MAR's OE is connected to logic 1 so it constantly outputs its value, since it contains the address of the current instruction, and it's very important to have access to that. The MAR's clock is connected to the inverted T1 from our timing signal generator (TSG) so it can read the program counter's (PC) value from the bus at the right moment.

The address bits A_0 to A_3 from the MAR go directly to the ROM, which outputs the instruction

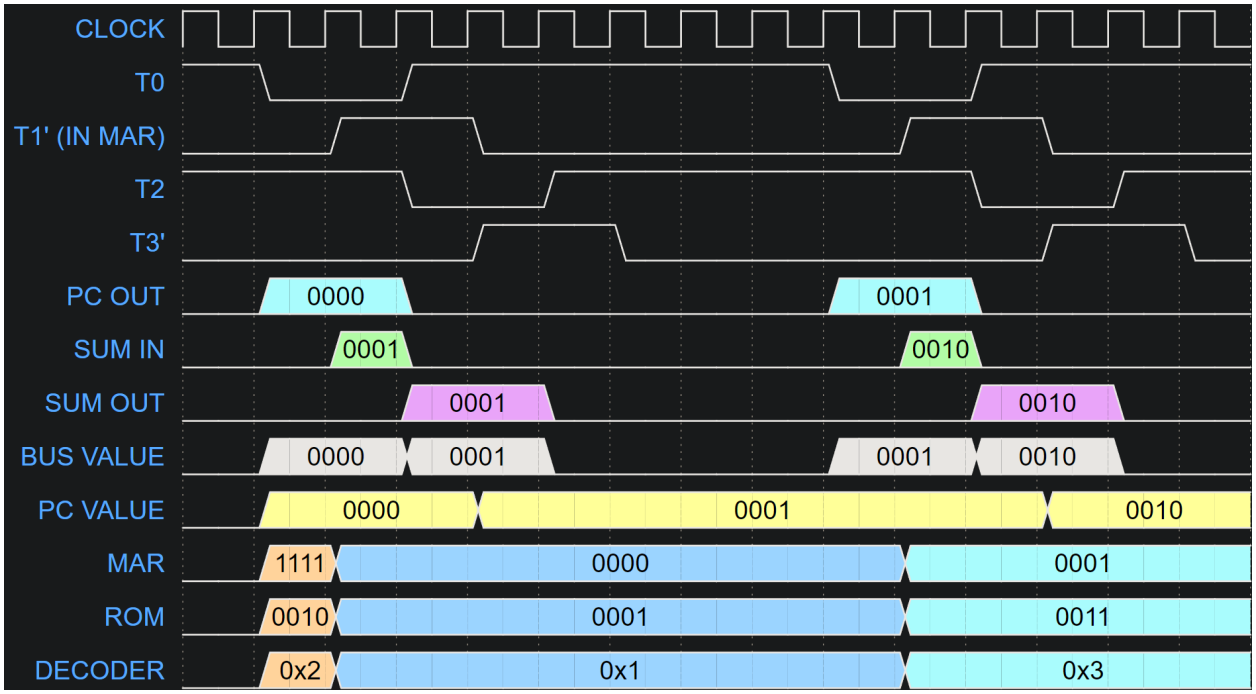


code in binary into the decoder which immediately decodes it and changes the appropriate control line to logic 0. We can load our custom program into the ROM with a bin file containing 16 numbers between 0 and 4 inclusively. Each number can range between 0 and 4 since our computer has only 5 instructions, and you can only put 16 numbers because our MAR loops between 0 and 15 (16 different addresses). By default, the ROM is filled with 0s.



The decoder's output is connected to LEDs. In the picture, the last instruction is selected, which is why its LED is off.

Here is our timing diagram:



CLOCK	5Hz clock of TSG
T0	Output bit 1 of TSG, linked to OE of PC register
T1' (IN MAR)	Inverted output bit 2 of TSG, linked to clock of sum register and MAR
T2	Output bit 3 of TSG, linked to OE of sum register
T3'	Inverted output bit 4 of TSG, linked to clock of PC register
PC OUT	Output of the PC register to the bus (read and incremented by the adder)
SUM IN	Input of the sum register from the adder (positive-edge triggered)
SUM OUT	Output of the sum register to the bus
BUS VALUE	Current value on the bus
PC VALUE	Value stored in the PC register
MAR	Current memory address outputted by the MAR
ROM	Memory value outputted by the ROM (depends on instruction file)
DECODER	Output of the decoder

1. Since the 74HC574 is able to store 8 bits of data, can it be used to replace two 4 bit registers? Explain your reasoning.

Two 4-bit registers can have two separate clocks and an 8-bit register can only have 1. The only way two 4-bit registers can be replaced by a single 8-bit register is if they are both connected to the same clock. If they are not connected to the same clock, replacing them with a single 8-bit register could damage the circuit and its components.

2. Can an MDR and MAR be written to at the same time in this circuit design? Explain your reasoning.

Both registers can access the bus at the same time since both of them are taking their information from the same lines (from the bus). In the project, when T1' is active (the timing signal for MAR), the MAR reads the value from the bus. Since the MDR always has access to the inputs, both the MAR and the MDR are simultaneously written to when t1' is true.

3. What would happen if the 74154 was replaced for another active low decoder, such as the 7442, which has only 10 outputs? If the op-code 0xFF would was provided by the ROM, What would happen?

In this lab, we were only required to have 4 outputs from the decoder. For this question let's assume that our 74LS154 chip has all of its outputs connected to the lights. The decoder could in theory have 16 different outputs or instructions. However, since the ROM only reads number by number, the ROM used in this lab could only output up to 9 (in binary). So, in this case, our 74LS154 would only have 10 different functional outputs.

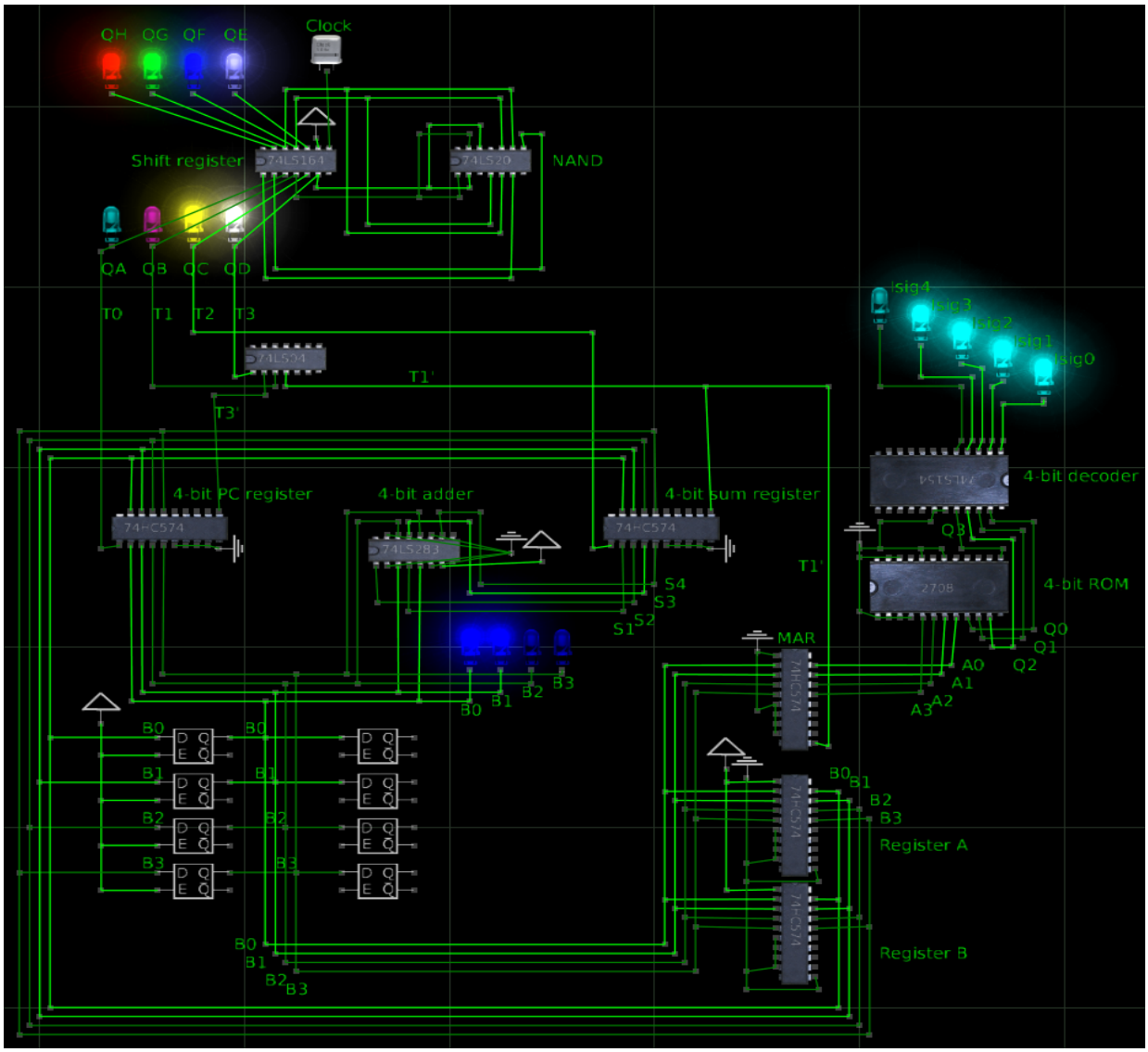
Now, let's replace the 74LS154 decoder with a 4-line to 10-line chip (7442). As stated above, our ROM can only output up to 9 in binary and our decoder could have a maximum of 10 operational instructions. Since the 7442 chip has 10 outputs and the same amount of inputs as the previous chip, there would be no difference between the result of the outputs of 74LS154 and the outputs of the 7442 chip.

For the second part, let's assume that our ROM chip did output its maximum value (0xF or 15 in binary). For the 74LS154 chip, this would trigger the last instruction (or lsig15). For the 7442 chip, nothing would happen as there is no pin for this type of input in our computer.

4. The address lines A9 to A4 are grounded, what would happen if one of those lines was changed to a logic 1. Explain your reasoning.

First, let's calculate the minimum address (A0 to A3 are 0): 2^4 +2^5 +2^6 +2^7 +2^8 +2^9=1008. So, 1008 would be the starting address of the ROM (also known as offset) which means that any number in the input file (from 0-9) up to the 1007th iteration would not have been read by the ROM. This example shows how important it is to ground the unused inputs of the ROM unless they are used as offsets. In other words, only the numbers from 1008 to 1023 would have been read by the ROM.

4. Discussion



It's becoming increasingly vital to organize our project since for now, the MAR is far away from the TSG, so the T1' line has to travel far to reach the MAR. In the simulation, this isn't important, but in real life, this wastes resources. Nevertheless, for this circuit, we have observed a high latency between cycles. This is mainly due to the TSG having the timing signals T4 to T7 unused. This is inefficient and leads to latency between each instruction read for now, but this leaves space in each cycle to execute the instruction in future projects. Furthermore, we have observed that the ROM cannot read numbers higher than 9, it will read them as separate numbers (e.g. 24 would be read as 2 and 4). This is weird since the ROM has enough pins to accommodate up to 16 different instructions. This is probably due to the bin file being read as a normal text file, where it assumes that each character is an instruction.

5. Conclusions

In this lab, we learnt about how to implement two data registers, a MAR, a ROM and a decoder. These components together can read and decode a simple bin file. Controlled by T1', the MAR reads a new address from the bus every clock signal. The address indicates to the ROM which memory location we want to access. The contents of the binary file are instructions read by the ROM and then decoded by the decoder into op-codes. Furthermore, we have seen how to use a binary file with integer inputs that range from 0 to 4 for a 4-bit ROM output (5 possible instructions, a sequence of 16 instructions). In the end, we have implemented our program counter with the 4-bit sum register and bus to be able to cycle through instructions. As our computer gets bigger, we will be able to perform different tasks with each of these instructions.